On this page

Change Log Levels at Runtime

Architectureâ□□

Feedzai's product is a distributed system with multiple components and each of those components is typically replicated. This can make investigating issues challenging and having more logs enabled could help in those investigations.

Typically changing log levels can be achieved by editing the local `logger.xml` files present in each machine but as we do not have access to them, an alternative solution was required. We've developed an API that runs in the Pulse Server which is able to affect log level changes, during runtime, across in each Pulse Server, Pulse Application and Case Manager instance.

This API writes a message to a new exchange in RabbitMQ (`hsbc-logging-sync`) and each component knows to read relevant messages from that queue and change their local log levels. The routing key of the message is the component which should affected by the log change.

On the consumer side, each Pulse Server, Pulse Application and Case Manager instance will have their own queues bound to this exchange and listening to particular routing keys. The table below lists the queues and bindings that are created for the exchange.

Component	Queue	Routing Key
Pulse Server	fdz-sq-hsbc-logging-sync-pulse-server-{host}	pulse-server
Pulse Application	fdz-sq-hsbc-logging-sync-pulse-application-{app_slug}-{host}	pulse-application:{app-slug}
Pulse Application	fdz-sq-hsbc-logging-sync-pulse-application-{app_slug}-{host}	pulse-application:*
Case Manager	fdz-sq-hsbc-logging-sync-case-manager-{host}	case-manager

All queues have the host in the queue name, ensuring that each replica will receive the messages. Each Pulse Application has two routing keys bound to their queue: one that affects all Pulse Applications and another specific to that app's slug.

The image below depicts what happens when a request to change a log level for a Pulse Application with slug `outbound` is sent to the API. First, upon receiving the request, the API will write a message to RabbitMQ using the routing key `pulse-application:outbound` . RabbitMQ will then route the message to the queues that are expecting that routing key. In this case, those would be `pulse-app-outbound-10.0.0.30` and `pulse-app-outbound-10.0.0.47` (note that the queue names have been simplified in this example). Eventually the consumers in each Pulse Application listening to those queues will notice the new messages and handle them by changing the log levels locally, according to the message payload.

The API lives in the Pulse Server and therefore inherits the authentication and authorization mechanisms there present.

What if Case Manager is connected to a different RabbitMQ cluster?

As the API lives in the Pulse Server, it inherits its RabbitMQ connection. However, it is possible to configure Case Manager to use a different RabbitMQ cluster. The API detects that by reading Pulse's Alert Storage Service configuration via the following properties:

- `pulse.global.services.AlertStorageService.dc.dc1.*`

If the brokers set in the Alert Storage Service global configuration are different from the ones configured for Pulse messaging, the API will connect to both clusters and route messages accordingly:

- any Pulse related requests will be written to the exchange in Pulse's RabbitMQ cluster
- any Case Manager related requests will be written to the exchange in Case Manager's RabbitMQ cluster

Loggers APIâ

The entrypoint to interact with this functionality is the Loggers API, available in each Pulse Server.

URL: `/admin/api/loggers/{targetComponent}/{loggerName}/{loggerLevel}`

Method: `POST` or `GET` (which allows the functionality to be accessed directly from the browser, using the permissions of the logged in user)

Authentication: This endpoint uses standard Pulse authentication. When using a `POST` request, a cookie must be sent in the header (see the API Authentication guide). When using a `GET` request via the browser, the user must have previously logged in via the Pulse UI in the same browser window.

Authorization: This endpoint required the logged in user to have the `security:logging:update` permission.

Parameters:

Name	Description
`targetComponent`	The component that should have the logger changed (this is used as the routing key in RabbitMQ). Valid values are: `pulse-server`, `pulse-application:{slug}`, `pulse-application:*` or `case-manager`.
`loggerName`	The logger name to be changed, for example `com.feedzai.hsbc.SomeClass`
`loggerLevel`	The logger level to be set. Valid values are `trace`, `debug`, `info`, `warn`, `error`, or `off` (case-insensitive)

Responses:

Code	Content-type	Response
`200`	`application/json`	OK - JSON that echoes back the parameters sent in the request.
`400`	`application/json`	Bad Request - JSON with the error message.
`401`	`text/html`	Unauthorized - Standard Jetty error page for unauthorized requests.
`403`	`application/json`	Forbidden - The logged in user is missing the required permissions.

Example using cURL:

```
curl -X POST http://<PULSE_UI>/admin/api/loggers/pulse-server/com.feedzai.pulse.service.jobmanager.exec.ExecutionManagerImpl/error \
  -H "Cookie: SSOcookie=cee99ab5-xxxx-xxxx-xxxx-xxxxxxxxxxxx" \
  -H "X-Requested-With: XMLHttpRequest"
```

Copy

Example response

```
{
  "component": "pulse-application:*",
  "loggerName": "com.feedzai.pulse.service.jobmanager.exec.ExecutionManagerImpl",
  "loggerLevel": "error"
}
```

Copy

Enabling Probe loggingâ

As a particularly useful case, the logging levels API can also be used to enable Pulse and Case Manager probes logging by setting specific probes-related loggers to log at `trace` level. Probes logging provides detailed timing information for incoming events and various other information that can be used in fine-grained performance debugging and analysis.

As probes logging creates a very large throughput of logging, which results in increased infrastructure and storage costs, and can even impact application performance, they should be used sparingly and only enabled for short periods of time when performing analysis and debugging, after which they should be changed back to `info` level.

Enabling Pulse Application probes for application with app slug (URL alias) `tfrb` - cURL

```
curl -X POST http://<PULSE_UI>/admin/api/loggers/pulse-application:tfrb/com.feedzai.metrics.probes.TextAbstractProbe/trace \
  -H "Cookie: SSOcookie=cee99ab5-xxxx-xxxx-xxxx-xxxxxxxxxxxx" \
  -H "X-Requested-With: XMLHttpRequest"
```

Copy

Disabling Pulse Application probes for application with app slug (URL alias) `tfrb` - cURL

```
curl -X POST http://<PULSE_UI>/admin/api/loggers/pulse-application:tfrb/com.feedzai.metrics.probes.TextAbstractProbe/off \
  -H "Cookie: SSOcookie=cee99ab5-xxxx-xxxx-xxxx-xxxxxxxxxxxx" \
  -H "X-Requested-With: XMLHttpRequest"
```

Copy

Enabling Case Manager probes - cURL

```
curl -X POST http://<PULSE_UI>/admin/api/loggers/case-manager/AlertManagerProbe/trace \
  -H "Cookie: SSOcookie=cee99ab5-xxxx-xxxx-xxxx-xxxxxxxxxxxx" \
  -H "X-Requested-With: XMLHttpRequest"
```

Copy

Disabling Case Manager probes - cURL

```
curl -X POST http://<PULSE_UI>/admin/api/loggers/case-manager/AlertManagerProbe/off \
  -H "Cookie: SSOcookie=cee99ab5-xxxx-xxxx-xxxx-xxxxxxxxxxxx" \
  -H "X-Requested-With: XMLHttpRequest"
```

Copy